PROJECT PROPOSAL

Daily Drink Water Reminder System

Course: Course: Software Engineering and Project Management

Submitted By:

Tanha Tabassum Tiha
ID: IT24048

Session: 2023-2024

Submitted To:

Md. Ziaur Rahman
Professor
Department of ICT



Mawlana Bhashani Science and Technology University

1. Abstract

The Daily Drink Water Reminder System is a software application designed to assist users in maintaining a regular water-drinking routine. Busy daily schedules frequently lead to inadequate fluid intake. This desktop application allows users to set daily water intake targets, log consumption in millilitres, receive automated periodic reminders, and monitor real-time daily progress.

Developed using standard Java and Java Swing, the application features an intuitive graphical user interface (GUI). Development follows the Incremental Software Development Model, delivering functionality through manageable iterations. The scope encompasses detailed requirement analysis, system architectural design, object-oriented implementation, testing, and comprehensive project documentation.

2. Introduction & Problem Statement

2.1 Introduction

Adequate daily water intake is essential for maintaining physical health, cognitive performance, and general well-being. However, students, office professionals, and individuals with demanding daily workloads frequently forget to consume sufficient water. Manual tracking using logs or memory is inconvenient and unreliable for calculating cumulative intake against personalized health targets. The Daily Drink Water Reminder System provides an automated solution that tracks fluid intake, calculates daily goals, and issues scheduled desktop reminders.

2.2 Problem Statement

In the absence of dedicated tracking tools, individuals struggle to maintain consistent hydration. Manual logging techniques suffer from high drop-off rates and lack real-time visual progress indicators. Existing comprehensive health suites are often overly complex or resource-heavy for simple desktop usage. Consequently, a lightweight application focused on scheduled reminders, quantitative intake tracking, progress visualization, and historical record keeping is required.

3. Objectives & Project Scope

3.1 Project Objectives

- **1.** Issue automated notifications at scheduled user-defined intervals.
- **2.** Provide functionality for configuring and updating daily intake goals (in ml).
- **3.** Facilitate quick-logging and custom volume entries for precise consumption tracking.
- **4.** Compute daily total consumption and remaining intake needed in real time.
- **5.** Display progress dynamically using visual indicators (percentage and progress bar).
- **6.** Retain historical consumption logs and handle automatic daily total resets.
- **7.** Implement an accessible, user-friendly graphical interface via Java Swing.

3.2 Scope Management

Included Features (In-Scope)	Excluded Features (Out-of-Scope)
User profile configuration (Name, Daily Goal)	Medical or clinical diagnostic advice
Quick-add buttons (150 ml, 250 ml, 500 ml)	Smartwatch/wearable hardware sync
Manual custom volume entry	AI-driven intake optimization
Automatic total and remaining fluid calculations	Cloud sync and online user accounts
Progress bar and percentage visualization	Background OS alerts when app is terminated
Customizable timer-based reminders	Multi-user networking infrastructure

4. Feasibility Study

- **Technical Feasibility:** Standard Java Development Kit (JDK) and Java Swing provide built-in visual controls and timer services (javax.swing.Timer). No specialized external hardware is required.
- **Economic Feasibility:** Built entirely with open-source tools (JDK, Git, VS Code/Eclipse), eliminating software licensing costs.
- **Operational Feasibility:** Designed with an intuitive single-dashboard layout, requiring zero formal end-user training.
- **Schedule Feasibility:** Structured across eight weeks using the Incremental Model to ensure milestone compliance and mitigate implementation risks.
- **Maintenance Feasibility:** Modular object-oriented design separates GUI, tracking logic, timers, and data storage into distinct classes.

5. Software Requirements Specification

5.1 Functional Requirements

ID	Requirement Description
FR-01	The system shall allow users to set and update their display name.
FR-02	The system shall allow users to configure a daily target volume in millilitres.
FR-03	The system shall allow users to record water consumption entries.
FR-04	The system shall compute total daily water intake instantly upon data entry.
FR-05	The system shall calculate the remaining intake towards the daily target.
FR-06	The system shall render real-time progress via a percentage indicator and visual bar.
FR-07	The system shall issue notifications at configured intervals while active.
FR-08	The system shall provide controls to toggle the reminder notification engine.
FR-09	The system shall display historical consumption entries in a structured log.
FR-10	The system shall reset daily totals at midnight while archiving historical records.

5.2 Non-Functional Requirements

- **Usability:** Clear layout, explicit button labeling, and high visual contrast.
- **Performance:** Dynamic UI responsiveness; calculations update in <100 ms.
- **Reliability:** Input validation to reject negative or non-numeric volume values.
- **Portability:** Operational across any platform hosting Java Runtime Environment (JRE 11+).

6. Technology Stack & Software Process Model

6.1 Proposed Technology Stack

Component	Technology Selected
Programming Language	Java (JDK 17+)
GUI Framework	Java Swing (javax.swing)
Reminder Engine	javax.swing.Timer / ScheduledExecutorService
Persistence Layer	Local File Storage (JSON/CSV) / In-Memory (MySQL/JDBC Optional)
IDE & Version Control	VS Code / Eclipse, Git & GitHub
Testing Framework	Manual Black-box Testing & JUnit 5

6.2 Incremental Software Development Model

The project adopts the Incremental Software Development Model. The system is engineered in distinct, fully functional iterations, allowing early verification of core components.

Increment	Target Features	Primary Output
Increment 1	Profile configuration, main layout, and goal setup	GUI Shell & User Profile
Increment 2	Consumption recording, total calculation, progress bar	Core Tracking Engine
Increment 3	Timer module, reminder options, notification alerts	Active Reminder Engine
Increment 4	History manager, full system integration, bug fixes	Integrated Application

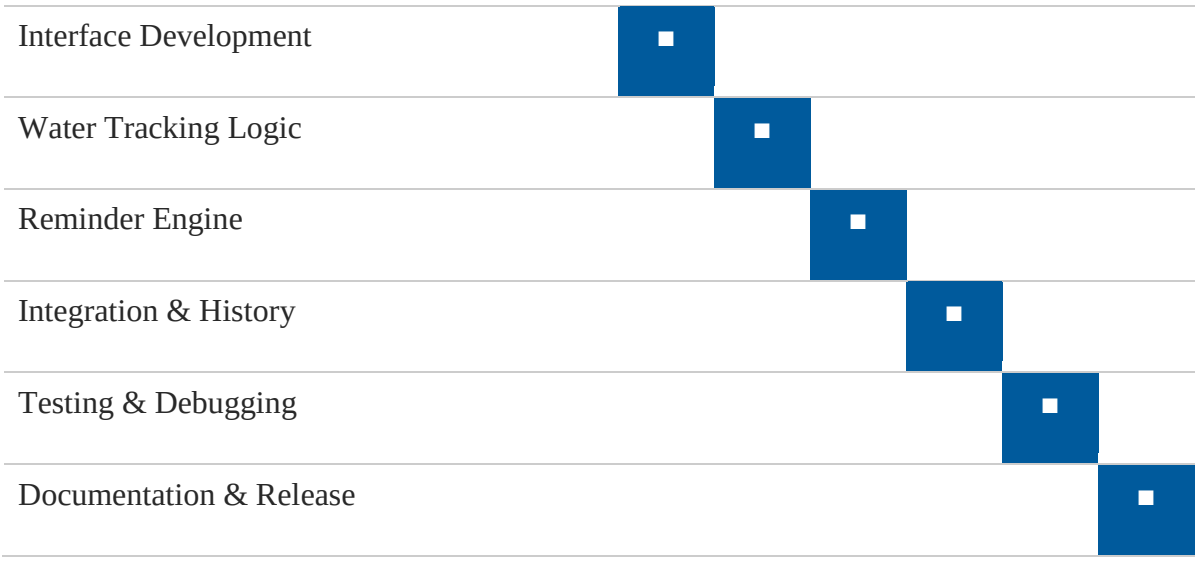
7. Project Schedule & Gantt Chart

7.1 Eight-Week Implementation Plan

Week	Primary Activities	Expected Deliverable
Week 1	Requirements gathering, feasibility, and scope definition	Requirement Specification
Week 2	System architecture design, DFD, ERD, and UML modeling	Design Documentation
Week 3	Interface layout construction and profile management	Increment 1 Release
Week 4	Core tracking calculations and quick-add logic	Increment 2 Release
Week 5	Reminder scheduling implementation and alert logic	Increment 3 Release
Week 6	Log history implementation and cross-module integration	Integrated Prototype
Week 7	System verification, functional testing, and bug fixing	Execution Test Report
Week 8	Final code refactoring, user guide, and submission	Final Deliverable Package

7.2 Gantt Chart Schedule

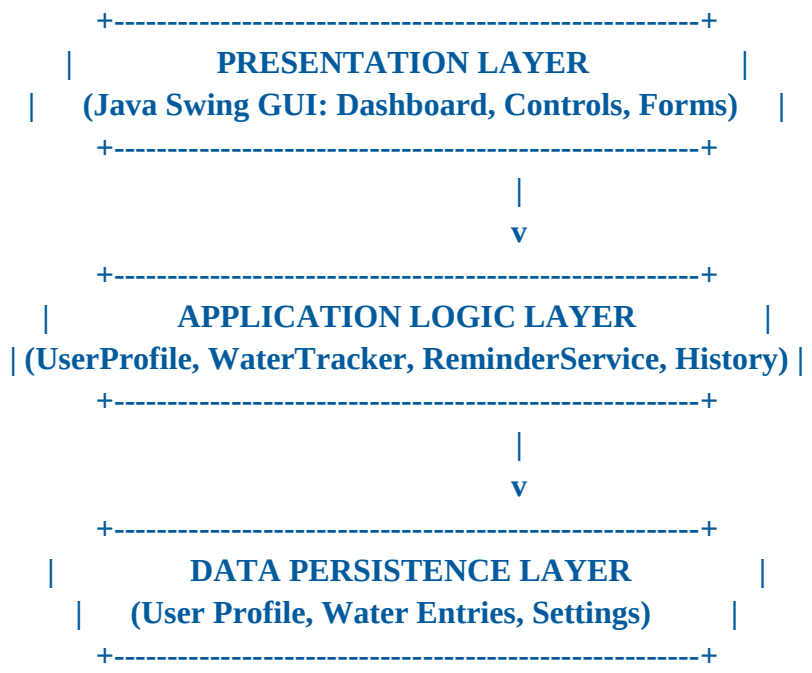
Activity	W1	W2	W3	W4	W5	W6	W7	W8
Requirement Analysis	■							
System & UML Design		■						


Interface Development
Water Tracking Logic
Reminder Engine
Integration & History
Testing & Debugging
Documentation & Release

8. System Architecture

The system adheres to a 3-Tier Logical Architecture:

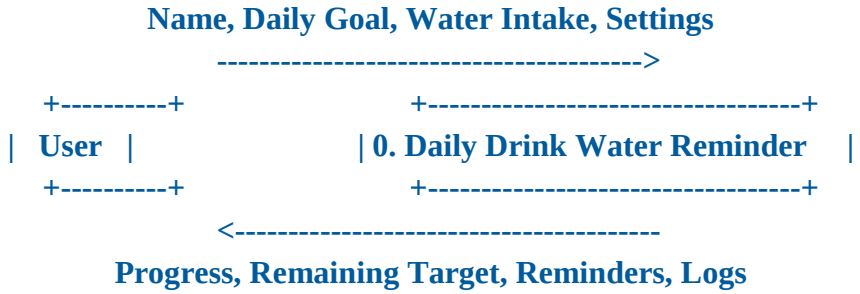
- **Presentation Layer:** Java Swing UI components handling user interaction, layout display, dashboard rendering, and user inputs.
- **Application Logic Layer:** Core business logic components including UserProfile, WaterTracker, ReminderService, and HistoryManager.
- **Data Persistence Layer:** Local file system or relational database storing user preferences, logs, and reminder parameters.



9. Data Flow Diagrams (DFD)

9.1 Level-0 Context Diagram

The Level-0 Context Diagram models the entire application as a central process interacting with the primary entity (User).



9.2 Level-1 Detailed DFD

- **Process 1.0 (Manage User Profile):** Receives profile input from User -> Writes to D1: User Profile.
- **Process 2.0 (Record Water Intake):** Receives intake volume -> Writes to D2: Water Records.
- **Process 3.0 (Calculate Daily Progress):** Reads D1 and D2 -> Computes progress & remaining volume -> Outputs to User.
- **Process 4.0 (Configure Reminders):** Receives timer preferences -> Writes to D3: Reminder Settings.
- **Process 5.0 (View History):** Reads D2: Water Records -> Formats historical logs -> Outputs to User.

10. Entity Relationship Diagram (ERD)

The database schema design consists of three primary entities:

- **USER_PROFILE:** PK user_id, name, daily_goal_ml
- **WATER_ENTRY:** PK entry_id, FK user_id, amount_ml, entry_time, entry_date
- **REMINDER_SETTING:** PK setting_id, FK user_id, interval_minutes, enabled_flag

Cardinality & Relationships

- **USER_PROFILE (1) -> (0..*) WATER_ENTRY:** One user records zero or more intake logs.
- **USER_PROFILE (1) -> (0..1) REMINDER_SETTING:** One user manages one set of reminder preferences.

11. Object-Oriented Design

11.1 Class Diagram Overview

The software structure comprises six distinct core classes:

- **WaterReminderApp:** Main application controller, handles Swing GUI execution and lifecycle.
- **UserProfile:** Stores user details (name, dailyGoalMl).
- **WaterEntry:** Represents individual logs (amountMl, entryTime).
- **WaterTracker:** Manages list of WaterEntry objects, calculates total volume and progress.
- **ReminderService:** Manages javax.swing.Timer execution and notification alerts.
- **HistoryManager:** Handles log retrieval, formatting, and file storage operations.

11.2 Object Diagram Instance Execution

An example instance runtime state for user 'Student':

- **profile1: UserProfile** -> name = "Student", dailyGoalMl = 2000
- **tracker1: WaterTracker** -> totalMl = 750
- **- entry1: WaterEntry** -> amountMl = 250
- **- entry2: WaterEntry** -> amountMl = 500
- **reminder1: ReminderService** -> intervalMinutes = 60, enabled = true
- **Computed Output** -> Remaining = 1250 ml, Progress = 37.5%

12. Testing Plan

Test ID	Scenario	Expected Output	Status
TC-01	Application Initialization	Main dashboard renders correctly	Pass
TC-02	Add Water (Quick Add 250 ml)	Total increases by 250 ml immediately	Pass
TC-03	Consecutive Intake Logging	Total correctly sums multiple logs	Pass
TC-04	Update Daily Goal (2500 ml)	Target and progress calculations update	Pass
TC-05	Compute Remaining Volume	Correctly displays Goal minus Total	Pass
TC-06	Progress Bar Update	Progress bar percentage updates dynamically	Pass
TC-07	Negative Input Validation	System rejects invalid values with warning	Pass
TC-08	View Log History	Stored historical entries are displayed	Pass
TC-09	Trigger Reminder Alert	Dialog popup appears at scheduled interval	Pass
TC-10	Disable Timer Controls	Timer stops; no further popups issued	Pass

13. Risk Analysis & Repository Structure

13.1 Risk Analysis Matrix

Identified Risk	Impact	Mitigation Strategy
Timer delay / Swing thread lock	High	Use javax.swing.Timer to maintain thread safety.
Data loss on application exit	Medium	Implement file saving on log entry and window closing.
Schedule slippage during integration	Medium	Strict adherence to 8-week incremental milestones.
Input format exceptions	Low	Strict validation logic using try-catch parsers.

13.2 GitHub Repository Structure

```
daily-drink-water-reminder/
├── README.md
├── src/
│   ├── WaterReminderApp.java
│   ├── UserProfile.java
│   ├── WaterEntry.java
│   ├── WaterTracker.java
│   ├── ReminderService.java
│   └── HistoryManager.java
├── diagrams/
│   ├── DFD-Level0.png
│   ├── UseCase.png
│   ├── ERD.png
│   └── ClassDiagram.png
└── docs/
    ├── PROJECT_PROPOSAL.pdf
    └── TEST_REPORT.md
```

14. Conclusion

The Daily Drink Water Reminder System addresses a real-world wellness problem by applying core software engineering practices. By following the Incremental Software Development Model and leveraging Java Swing, the project delivers a functional desktop application within an 8-week timeframe. The project demonstrates full lifecycle implementation including requirements engineering, object-oriented design, modular coding, visual testing, and comprehensive project documentation.